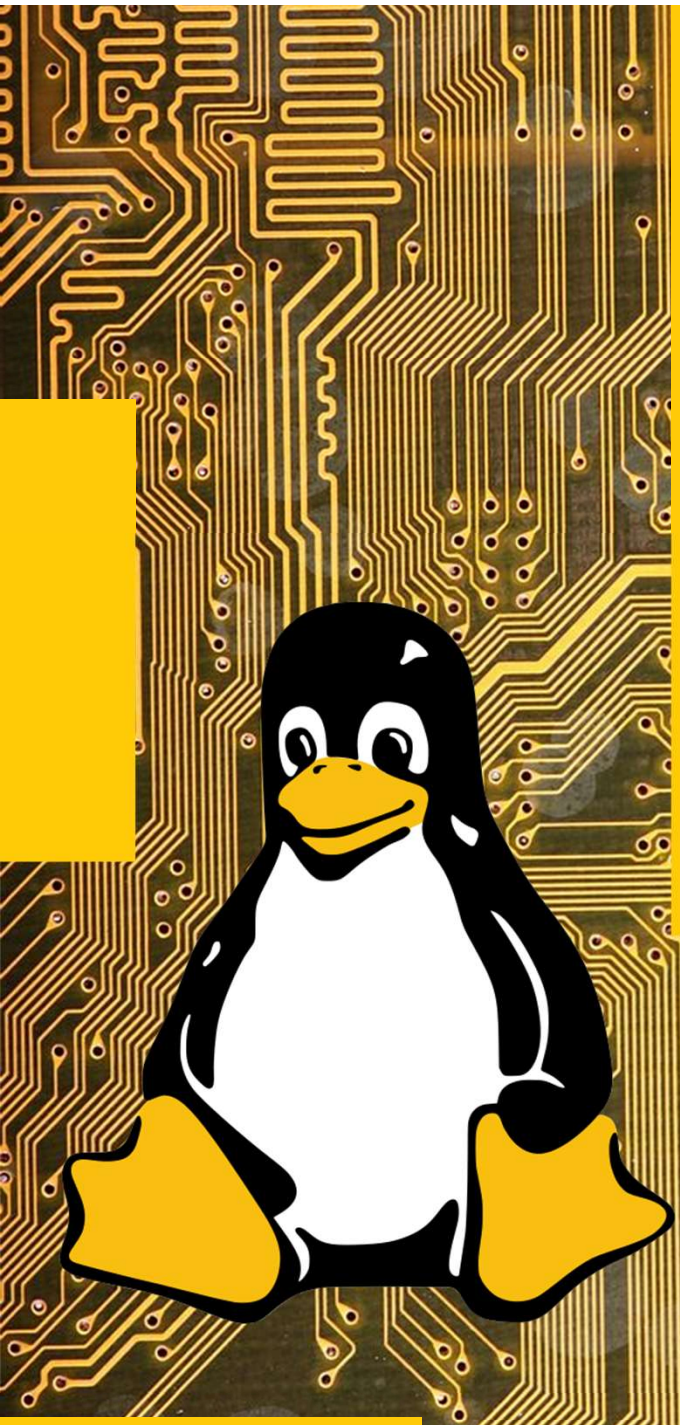


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

IPC II

Semaphores

INSTRUCTOR: Jalolliddin Yusupov



WEEK IV Course Topics & Weekly Lecture Breakdown

Lecture Topic: IPC - Message Queues, Shared Memory and Semaphores (TLPI Ch 45-54)

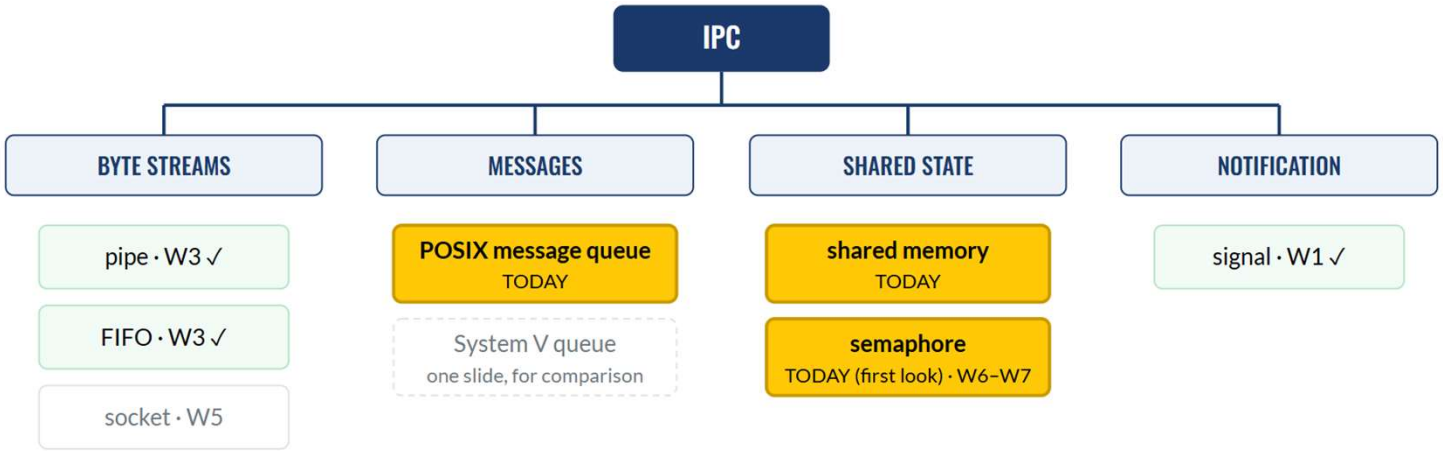
- Message queues: POSIX and System V interfaces, message priorities, the persistence of System V objects. Shared memory: creating and mapping a shared region. The lost-update problem demonstrated, and the first semaphore as the lock that protects shared memory between processes. Cleaning up IPC objects.

WHERE WE ARE GOING

Two more ways to talk: the message queue (a mailbox) and shared memory (a shared fridge) — what they are, when to use them, what they cost

The lost update: two processes add 1 a million times each, and the answer is wrong. What we need: mutual exclusion, a **mutex**

Semaphores: how to trace it on paper, and three ways to use it — a lock, a limit, a signal



Reading

TLPI ch. 51–54 (POSIX IPC, message queues, semaphores, shared memory).
Silberschatz §3.5, §6.6.
Manual pages:
mq_overview(7),
shm_overview(7),
sem_overview(7).

Three Things a Lock Cannot Say

unlock(L)

free: you can access the critical section



mutex
Mutual EXclusion

taken: flag is missing, you need to wait

lock(L)



A mutex lock has only two states: **free** or **taken**

VS

Real programs need more

1 · "At most 3 at a time"

A server allows 3 clients to use the database at once, not 1. The 4th must wait until one leaves. A lock can count only to 1.

2 · "B only after A"

The child may read the file only **after** the parent has written it. That is not about sharing — it is about **order**. With a lock, whoever comes first wins.

3 · "Take turns"

PING, PONG, PING, PONG … forever, never two PINGs in a row. A lock guarantees one at a time, not **whose** turn it is.

```
// Shared variable
Lock L = new Lock()
```

```
Process P1:
    lock(L)
    print("A")
    print("B")
    unlock(L)

Process P2:
    lock(L)
    print("C")
    print("D")
    unlock(L)
```

```
Process P3:
    lock(L)
    print("C")
    print("D")
    unlock(L)
```

What all three need?

```
Process P1:
    lock(L)
    print("A")
    print("B")
    unlock(L)

Process P2:
    lock(L)
    print("C")
    print("D")
    unlock(L)
```

```
Process P1:
while (true) {
    lock(L)
    print("ping")
    unlock(L)
}
Process P2:
while (true) {
    lock(L)
    print("pong")
    unlock(L)
}
```

The Semaphore

A **counter** that processes can decrease and increase **atomically**, and that makes a process **sleep** when there is nothing left to take. That counter is the **semaphore**.

- mutex
  - lock/unlock
  - acquire/release
- Semaphore
  - wait/post
  - P/V
  - wait/signal

synonymous terms

A **semaphore** S is an integer that is **never negative**, with exactly two (atomic) operations:

S=3

```
wait(S):    if S > 0:   S = S - 1
            else:       sleep until S > 0,
                        then S = S - 1

post(S):    S = S + 1
            if someone sleeps on S:
                        wake ONE of them
```



How to read the drawing

The **value** = the number of flags in the "hole". **wait** takes a flag; if the "hole" is empty the process **sleeps** beside it. **post** puts a flag back — or hands it directly to a sleeper, who wakes up.

The Trace Table

S STARTS AT 1 · THREE PROCESSES P, Q, R

#	operation	S	sleeping	inside
0	(start)	1	—	—
1	P: wait(S)	0	—	P
2	Q: wait(S)	0	**Q**	P
3	R: wait(S)	0	**Q, R**	P
4	P: post(S)	0 (flag goes to Q or R)	R	Q
5	Q: post(S)	0 (flag went to R)	—	R
6	R: post(S)	1	—	—

Four columns, one row per step

operation · the **value** after it · who is **sleeping** · who is **inside** (has passed wait and not yet posted).

The value can never go below 0: you cannot take a flag that is not there. A sleeper costs **no CPU**: it is not checking the "hole" in a loop, the OS wakes it (state S in ps).

The only things you may do

Set the start value **once**. After that: only wait and post. You may **not** read the value to decide something — it can change the moment after you read it.

Homework: Trace the Value

S STARTS AT 2 · THREE PROCESSES P1, P2, P3

#	operation	S	sleeping	inside
1	P1: wait(S)			
2	P2: wait(S)			
3	P3: wait(S)			
4	P1: post(S)			
5	P4: wait(S)			
6	P2: post(S)			
7	P3: post(S)			
8	P4: post(S)			

S = 1 — Start at 1: Your Semaphore is a Mutex

LAST LECTURE

```
Process P1:          Process P2:
  lock(L)              lock(L)
  print("A")           print("C")
  print("B")           print("D")
  unlock(L)            unlock(L)
```

TODAY — S STARTS AT 1

```
Process P1:          Process P2:
  wait(S)              wait(S)
  print("A")           print("C")
  print("B")           print("D")
  post(S)              post(S)
```

Same code, different start value, different program.

The same program

One flag in the "hole" = one key. wait takes the key, post gives it back. A semaphore that starts at **1** and is used as wait … post by the **same** process **is** a mutex. Its value is only ever 0 or 1: a **binary semaphore**

Answer three times:

(a) S starts at **1**

(b) S starts at **2**

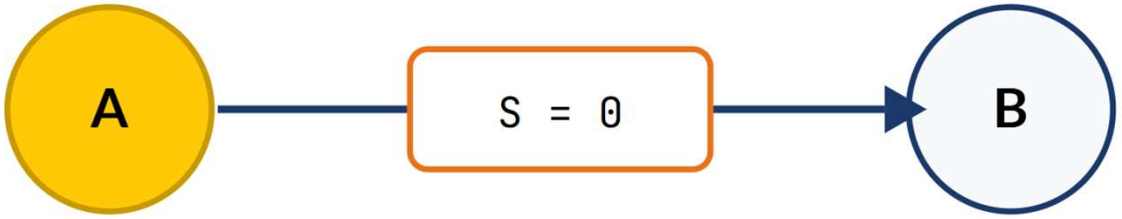
(c) S starts at **0**

start	possible outputs
S = 1	only **A B C D** and **C D A B**
S = 2	**all four** — and also A C D B, C A B D
S = 0	**none** — nothing is printed

Start at 0: "A Must Happen Before B"

```
S = 0                       /* nothing happened yet */

Process P1:                 Process P2:
  print("A")                  wait(S)
  post(S)                     print("B")
```



A: post(S) after

B: wait(S) before

Why it works

The golf "hole" starts **empty**. If P2 comes first, its wait finds no flags and it **sleeps**. The only flag ever created is the one P1 puts in **after** printing A. So B can never come before A — whatever the scheduler does.

Why not just `sleep(1)` before B?

Because on a busy machine A may take 2 seconds. `sleep` guesses; a semaphore **knows**.

Try It on Paper: Force the Order A, B, C

Three processes. The output must **always** be **A B C**, whatever the scheduler does. Fill in the blanks with wait(...) or post(...) and choose the starting values.

```
semaphores:  s1 = ___    s2 = ___

P_A:            P_B:            P_C:

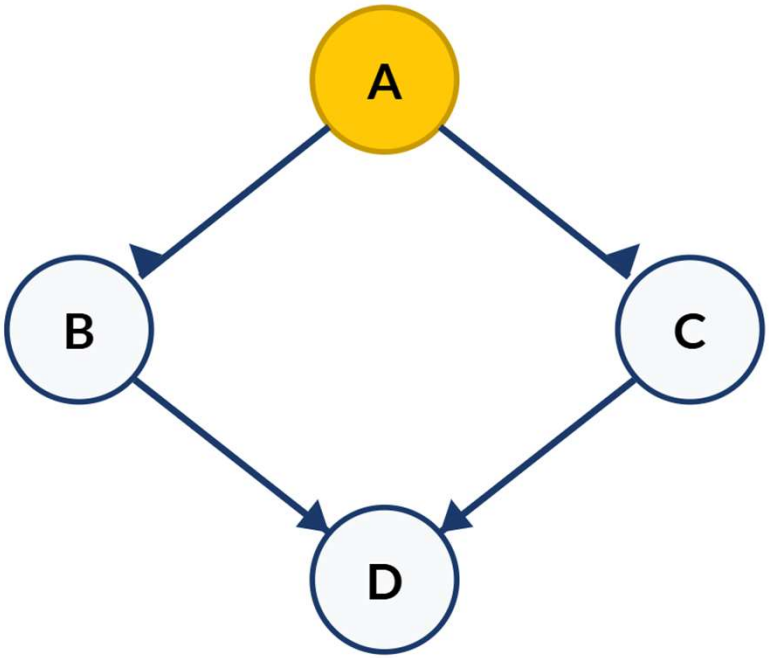
  --------        --------        --------
  print("A")      print("B")      print("C")
  --------        --------        --------
```

Some blanks stay empty. Use as few semaphores as possible.

```
s1 = 0    s2 = 0

P_A:            P_B:            P_C:
                  wait(s1)        wait(s2)
  print("A")      print("B")      print("C")
  post(s1)        post(s2)
```

From a Drawing to a Program



An arrow X → Y means: **X must be printed before Y**.

Four processes print one letter each: PA prints A, PB prints B, PC prints C, PD prints D.

(a) How many semaphores? What start values?

(b) Write the four processes.

(c) List every output that is possible.

Hint

A process with two arrows coming **in** waits twice.

(a) One semaphore per arrow, all start at **0**:
sAB, sAC, sBD, sCD.

```
Process PA:
  print("A")
  post(sAB)
  post(sAC)
```

```
Process PB:
  wait(sAB)
  print("B")
  post(sBD)
```

```
Process PC:
  wait(sAC)
  print("C")
  post(sCD)
```

```
Process PD:
  wait(sBD)
  wait(sCD)
  print("D")
```

In C: Four Calls

```c
#include <semaphore.h>
int sem_init   (sem_t *s, int pshared, unsigned int value);  /* once, before fork   */
int sem_wait   (sem_t *s);          /* P: take one, or sleep until you can        */
int sem_post   (sem_t *s);          /* V: give one back, wake one sleeper         */
int sem_destroy(sem_t *s);          /* at the end                                 */
```

Two new words

sem_t — the semaphore type. You never look inside it; you only pass its address to the four calls.
pshared — "shared between processes?" **1** = yes (always, in this course). Books often write 0: that means threads only.

WHERE THE SEMAPHORE LIVES

```c
struct shared {
    sem_t lock;      /* the "hole" ...          */
    long  counter;   /* ... and number of flags */
};
struct shared *s = shared_alloc(sizeof(struct shared));
sem_init(&s→lock, 1, 1);
```

The semaphore sits **in shared memory, next to the data it guards**, so both processes use the same one.

Four calls: init once, wait before, post after, destroy at the end.

Run it on Linux

macOS does not implement sem_init (it fails). Use your Ubuntu machine or VM. Compile with -pthread.

The Counter, With a Semaphore

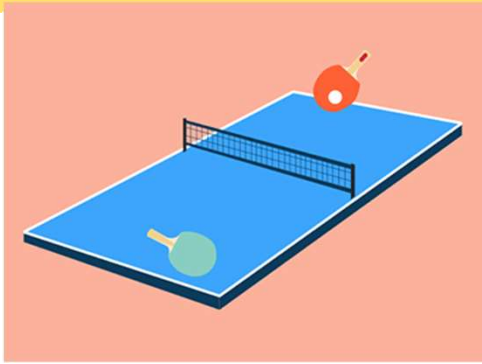
```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <semaphore.h>
#include "shared_mem.h"
#define N 1000000
struct shared {
    sem_t lock;  /* the padlock ...               */
    long  counter; /* ... and what it protects    */
};

int main(void) {
    struct shared *s = shared_alloc(sizeof(struct shared));
    s->counter = 0;
    sem_init(&s->lock, 1, 1); /* 1: shared by processes, 1: open */
    pid_t pid = fork();
    for (int i = 0; i < N; i++) {
        sem_wait(&s->lock); /* take the key, or sleep          */
        s->counter++;   /* critical section                */
        sem_post(&s->lock); /* give the key back              */
    }
    if (pid == 0) exit(0);
    wait(NULL);
    printf("expected %d, got %ld\n", 2 * N, s->counter);
    sem_destroy(&s->lock);
    return 0;
}
```

You need the -pthread flag because POSIX **semaphores** are part of the POSIX Threads library (**libpthread**)

```
gcc -o sem sem.c -pthread
```

The Counter, With a Semaphore



```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <semaphore.h>
#include "shared_mem.h"
#define N 1000000
struct shared {
    sem_t lock;  /* the padlock ...               */
    long  counter; /* ... and what it protects    */
};
```

```c
struct shared { sem_t ping; sem_t pong; };
```

```c
int main(void) {
    struct shared *s = shared_alloc(sizeof(struct shared));
    s->counter = 0;
    sem_init(&s->lock, 1, 1);
    pid_t pid = fork();
    for (int i = 0; i < N; i++) {
        sem_wait(&s->lock); /* take the key, or sleep          */
        s->counter++;   /* critical section                 */
        sem_post(&s->lock); /* give the key back              */
    }
    if (pid == 0) exit(0);
    wait(NULL);
    printf("expected %d, got %ld\n", 2 * N, s->counter);
    sem_destroy(&s->lock);
    return 0;
}
```

```c
sem_init(&s->ping, 1, 1);
sem_init(&s->pong, 1, 0);
```

```
gcc -o sem sem.c -pthread
```



Questions?